

B.Tech. (Information Technology)

Python Lab (6IT4-23)



By – Vinod Yadav

Assistant Professor

Department of Computer Science and Engineering

SKIT, Jaipur

1. Introduction

1.1 What are Programming Languages

- A programming language is a way to communicate with a computer.
- We give instructions to the computer using these languages.
- Python is one such programming language that is simple and powerful.
- Python is a high-level, general-purpose programming language.
- It is used to build websites, software, automate tasks, and analyze data.

1.2 Translator

- Computers cannot understand high-level programming languages like Python, C, or Java directly.
- It only understands binary language (0s and 1s).
- That's why we need a translator to convert our code into binary form.

For Example:

Just like we use a translator to convert English into Hindi while talking to someone who only understands Hindi, computers also need a translator to understand programming languages.

Types of Translators

There are mainly three types of translators used in programming:

1. Compiler

- A compiler converts the entire program into machine code in one go.
 - It checks all the errors after compilation.
- Example: C, C++ use compiler.

2. Interpreter

- An interpreter reads and executes the code line by line.
 - It stops immediately when it finds an error.
 - It helps in testing and debugging code easily.
- Example: Python, JavaScript use interpreter.

3. Assembler

- Assembler converts assembly language code into machine language.

- It is mainly used in low-level programming.

Why Python is called an Interpreted Language?

- Python uses an interpreter to execute its code.
- It reads and runs code line by line.
- That's why Python is known as an interpreted language.
- This makes it easy to test, debug, and learn.

1.3 Features of Python

- Python provides many powerful features that make it one of the most popular programming languages in the world.

1. Easy to Learn

- Python has a very simple and clean syntax.
- Beginners can learn it quickly.

2. Open Source

- Python is freely available for everyone.
- Anyone can download, use, and modify it.

3. High-Level Language

- Python code is close to human language, not machine language.
- This makes writing programs easier.

4. Object-Oriented Language

- Python supports classes, objects, inheritance, and polymorphism.
- This helps in writing organized and reusable code.

5. Portable

- Python code can run on different operating systems without changes.
Example: Windows code can run on Linux easily.

6. Platform Independent

- Write once, run anywhere.
Python works on Windows, Mac, Linux, etc.

1.4 History of Python

- Python was developed by Guido van Rossum.
- It started as a hobby project in the late 1980s.
- Guido wanted to create a simple and powerful programming language.
- He started developing Python in 1989 at Centrum Wiskunde & Informatica (CWI).
- Python was officially released in 1991.
- The first version was Python 0.9.0.
- Python 2.0 was released in 2000.
- Python 3.0 was released in 2008.
- Python is one of the most popular languages in the world.
- It is used in AI, ML, Web Development, Data Science, Robotics, etc.



2. Basics of Python Programming

2.1 First Python Program

- Python programming starts by writing simple statements.
- Let's print "**Hello, World!**" — the classic first program.

```
print("Hello, World!")
```

Output:

Hello, World!

This code prints a message to the screen.

2.2 Comments in Python

- Comments are notes written in the code that are ignored by Python during execution.
- They help programmers understand the code easily.

1. Single-line Comment

Program:

```
# This is a single-line comment  
print("Hello World")
```

We use # at the beginning of a single-line comment.

2. Multi-line Comment

Program:

```
'''  
This is a multi-line comment.  
It can span multiple lines.  
'''  
print("Hello, World!")
```

We use triple quotes (''' or """) for multi-line comments.

2.3 Python Indentation

- Indentation means spaces at the beginning of a line.

- Python uses indentation to define blocks of code.

Example (Wrong Indentation)

```
if 1:  
print("Happy") # Error
```

Without correct indentation, Python will give an error.

Example (Correct Indentation)

```
if 1:  
    print("Happy") # Correct
```

2.4 Variables in Python

- A variable is a name given to a memory location where data is stored.
- It is used to store, retrieve, and update information during program execution.
- Each variable stores a value of a different type.
- In Python, we do not need to declare the data type before using a variable.
- When you assign a value to a variable, Python stores it in memory and the variable name acts as a reference.

Example

```
name = "Ram"  
age = 19  
marks = 98.79
```

Variable Assignment Techniques

1. Single Assignment

Assign one value to one variable.

```
x = 10
```

2. Multiple Assignment

Assign values to multiple variables in a single line.

```
a, b, c = 1, 2, 3
```

3. One Value to Multiple Variables

Assign the same value to multiple variables.

```
x = y = z = 100
```

type() Method

type() is a built-in function in Python.

It is used to find out the data type of any variable or object.

Program:

```
print(type(name)) # <class 'str'>
print(type(age))  # <class 'int'>
```

Rules for Naming Variables

1. Variable name must start with a letter or underscore _.
2. It cannot start with a number.
3. Only letters, numbers, and underscores are allowed.
4. Variable names are case-sensitive (age and Age are different).
5. Keywords (like if, while, def) cannot be used as variable names.

2.5 Character Set

- Character set is the collection of valid characters that Python recognizes and uses to write programs.
- In simple words, these are the characters allowed in Python code.

Types of Character Set

1. Letters

Letters include uppercase and lowercase English alphabets.

- Uppercase: **A to Z**
- Lowercase: **a to z**

Letters are used to create variable names, function names, and keywords.

Example:

```
name, totalMarks, print
```

2. Digits

- Digits include numbers from **0 to 9**.
- Digits are used to represent constants and numeric literals.

Example:

10, 25, 2025, 3.14

3. Special Characters

Special characters are symbols used to perform operations.

Examples:

+ - * / % = < > () [] { } , : . ; @ #

4. White Spaces

- White spaces include space, tab, and newline.
- They are used to separate tokens and for indentation.

Example:

```
if a > b:  
    print(a)
```

5. Escape Sequences

- Escape sequences are special characters written using backslash \.
- They are mainly used inside strings.
- \n → new line
- \t → tab space
- \\ → backslash
- \' → single quote
- \" → double quote

6. Unicode Characters

Python supports Unicode characters, so it can work with characters of many languages.

- ord() returns the Unicode value of a character.
- chr() returns the character from a Unicode value.

Example:


```
print(ord('A')) # 65  
print(chr(65))  # A
```



3. Tokens

- Tokens are the smallest units in a Python program that carry meaning.
- Python breaks every line of code into tokens while compiling/interpreting.

Types of Tokens:

- Keywords
- Identifiers
- Literals
- Operators
- Punctuators

3.1 Keywords

- Keywords are special reserved words that have predefined meanings in Python.
- The compiler/interpreter already knows their purpose.
- Keywords cannot be used as identifiers (variable names, function names).

Total Keywords – 35

List of Keywords:

False, await, else, import, pass, None, break, except, in, raise, True, class, finally, is, return, and, continue, for, lambda, try, as, def, from, nonlocal, while, assert, del, global, not, with, async, elif, if, or, yield

3.2 Identifiers

- Identifiers are names used for variables, functions, classes, etc.
- They are defined by the programmer.

Example:

```
name = "ram" # name is an identifier
```

3.3 Literals

Literals are fixed values assigned to variables in the program.

Types of Literals:

1. Numeric Literals
2. String Literals
3. Boolean Literals

4. Special Literal
5. Literal Collections

1. Numeric Literals

Numeric literals are fixed number values used directly in the code.

Types of Numeric Literals:

- a) Integer Literal
- b) Float Literal
- c) Complex Literal

a) Integer Literal:

An integer literal represents a whole number without any decimal.

Example:

```
a = 10  
b = -45  
c = 0
```

b) Float Literals:

Float literals represent real numbers with decimal points.

Example:

```
a = 8.9  
b = -19.8
```

c) Complex Literals:

Complex literals are numbers with real and imaginary parts. They are written in the form: $a + bj$

Example:

```
a = 2 + 3j
```

Here, 2 is the real part and 3j is the imaginary part.

2. String Literals

A string literal is a sequence of characters enclosed within quotes.

1. Single-Line String Literal:

Written in a single line using single or double quotes.

```
m1 = "Hello, World!"
```

2. Multi-Line String Literal:

Written using triple quotes (''' or ''').

```
m1 = ''' Welcome to SKIT '''
```

or

```
m1 = " Welcome to SKIT "
```

3. Empty String Literal:

An empty string means no characters are present.

```
m1 = ""
```

It is useful when you want to initialize a string with no value.

3. Boolean Literals

Boolean literals represent the truth values: True or False. They are used in conditions, decision making, loops, and comparisons.

Example:

```
if True:  
    print("Happy")
```

4. Special Literal

Python has a special literal called None. It represents the absence of a value.

Example:

```
m1 = None
```

- Keyword: None
- Data Type: <class 'NoneType'>

5. Literal Collection

Collection literals are used to store multiple values in a single variable. They help in grouping data types like numbers, strings, or combinations.

- List: [1, 2, 3, 4]
- Tuple: (10, 20, 30)
- Dictionary: {'name': 'ram', 'age': 20}
- Set: {1, 2, 3, 4}

3.4 Operators

An operator is a symbol that performs an operation on variables or values.

Example:

```
a = 10 + 5 # '+' is an operator
```

Precedence and Associativity Rules

Precedence means the priority of operators when multiple operators appear in an expression.

```
a = 10 + 5 * 2  
print(a) # Output: 20
```

* has higher precedence than +.

Associativity decides the direction of execution when operators have the same precedence.

```
a = 5 - 2 + 1  
print(a)
```

Evaluated from left to right.

Operator Precedence Table

Level	Operators	Associativity
1	()	Left to Right
2	**	Right to Left
3	+, - (unary)	Right to Left
4	*, /, //, %	Left to Right
5	+, - (binary)	Left to Right

6	==, !=, >, <, >=, <=	Left to Right
7	not	Right to Left
8	and	Left to Right
9	or	Left to Right
10	=, +=, -=, etc	Right to Left

Types of Operators

1. Arithmetic Operators
2. Relational / Comparison Operators
3. Logical Operators
4. Assignment Operators
5. Identity Operators
6. Membership Operators

1. Arithmetic Operators

Used for mathematical calculations.

Operators: + - * / // % **

Example:

```
a = 10
b = 5
print(a + b)
print(a - b)
print(a * b)
print(a / b)
print(a // b)
print(a ** b)
```

2. Relational / Comparison Operators

Used to compare two values.

They return either True or False.

Operators: ==, !=, >, <, >=, <=

3. Logical Operators

Used to combine multiple conditions.

Operators:

- not
- and
- or

4. Assignment Operators

Used to assign values to variables.

Operators: = += -= *= /= //= %= **=

5. Identity Operators

Used to compare memory locations of two objects.

Operators:

- is
- is not

6. Membership Operators

Used to check whether a value is present in a sequence.

Operators:

- in
- not in

Example:

```
fruits = ["apple", "banana", "cherry"]  
print("banana" in fruits)
```

```
nums = [10, 20, 30]  
print(40 not in nums)
```

3.5 Punctuators

Punctuators are symbols used in Python programming to organize the structure and meaning of code.

Examples:

() {} [] : , ; #

4. Data Types

- Data types define the type of value a variable can hold.
- Python is dynamically typed, which means you do not need to declare the data type explicitly; Python identifies it automatically.

4.1 Types of Data Types

1. Numeric Type (int, float, complex)
2. Boolean
3. Sequence Types
 - String
 - List
 - Tuple
4. Mapping Type (Dictionary)
5. Set Types (set)
6. None Type

1. Numeric Types

i) int

Used to store whole numbers (positive or negative).

Example:

a = 10

ii) float

Used to store decimal numbers.

Example:

a = 6.78

iii) complex

Used to store complex numbers in the form $a + bj$.

Example:

A = 3 + 2j

2. Boolean Type

Boolean type has only two values: True and False.

Example:

```
A = True  
B = False
```

3. Sequence Types

(i) String (str)

A string is a sequence of characters enclosed within single, double, or triple quotes.

Example:

```
Name = "Ram"
```

(ii) List (list)

Lists are ordered, mutable (changeable), and allow duplicate values.

Example:

```
fruits = ["apple", "banana", "cherry"]
```

(iii) Tuple (tuple)

Tuples are ordered, immutable (not changeable), and allow duplicate values.

Example:

```
colors = ("red", "green", "blue")
```

4. Mapping Type – Dictionary (dict)

A dictionary stores data in key-value pairs.

Example:

```
student = {"name": "Ram", "age": 21}
```

5. Set Types (set)

Sets are unordered and do not allow duplicate values.

Example:

```
nums = {1, 2, 3}
```

6. None Type

Represents the absence of a value or a null value.

Example:

```
x = None  
print(type(x)) # <class 'NoneType'>
```

4.2 Immutable vs Mutable Data Types**i) Immutable Data Types**

- Immutable data types cannot be changed after creation.
- If you try to modify them, a new object is created instead.

Examples: int, float, str, complex, tuple

Program:

```
x = 10  
x = x + 5 # Old value not modified, new object created  
  
name = "Ram"  
# name[0] = "J" # Error – strings are immutable
```

ii) Mutable Data Types

- Mutable data types can be changed after creation.
- Changes happen in the same memory location.

Examples: list, dict, set

Program:

```
mylist = [1, 2]  
mylist.append(3) # Modified without creating a new object
```

5. Input and Output Operations

- Input means taking data from the user during program execution.
- Output means displaying result or message to the user.

5.1 input() Function:

- input() is a built-in function used to take input from the user.

Program:

```
name = input("Enter your name: ")
print("Hello", name)
print(type(name)) # output: <class 'str'>
```

Note:- By default, input is taken as a string.

Example with type conversion (casting):

```
age = int(input("Enter your age: "))
price = float(input("Enter price: "))
```

Since input is always a string, we must convert to number if needed.

Note:

If a number is required, conversion using int() or float() is necessary.

5.2 print() Function:

- print() is used to show output on the screen.

Program:

```
print("Welcome to SKIT")
```

It can print:

- Strings ("Hello")
- Numbers (5, 3.14)
- Variables (name, age)
- Expressions (10 + 20), etc.

sep and end in print():

These are optional arguments in print() to control output format.

i) sep:

Separator between values.

Example:

```
print("A", "B", "C", sep="-") # Output: A-B-C
```

ii) end:

What to print at the end of the line (default is \n, i.e., newline).

Example:

```
print("Hello", end=" ")  
print("World") # Output: Hello World
```

5.3 format() Function:

format() method is used to insert values inside a string using placeholders {}.

Example:

```
name = "Vinod"  
age = 20  
print("My name is {} and I am {} years old".format(name, age))
```

Positional Indexing Example:

```
print("Age: {1}, Name: {0}".format("Vinod", 20))
```

You can choose which value goes where.

5.4 f-Strings (Formatted Strings):

An f-string (formatted string) is a way to embed variables or expressions directly inside a string using {}.

Example 1:

```
name = "Vinod"  
print(f"Hello {name}") # output: Hello Vinod
```

Example 2:

```
a = 10  
b = 5  
print(f'Sum = {a + b}, Product = {a * b}')    # output: Sum = 15, Product = 50
```



6. Type Casting

- Type casting means converting a value from one data type to another.
- We use type casting to perform operations between different data types correctly.

Example:

Whenever we take user input in Python, it is always received as a string, even if the user types a number. Because of this, we cannot perform mathematical operations directly on that input. To perform calculations, we must convert the string into numeric types like int or float.

Program:

```
age = input("Enter your age: ")    # age is a string ("20")
age = int(age)                    # convert string to integer
print(age + 5)
```

6.1 Types of Type Casting:

1. Implicit Type Casting
2. Explicit Type Casting

1. Implicit Type Casting:

- When Python automatically converts one data type into another without the programmer doing anything, it is called implicit type casting.
- Python automatically promotes lower data type to higher data type.

Example:

```
a = 5        # int
b = 2.5      # float
c = a + b    # int + float → float
print(c)     # Output: 7.5
```

2. Explicit Type Casting:

When the programmer manually converts one data type into another using predefined Python functions, it is called explicit type casting.

Common Functions:

- `int()` → Convert to integer

- float() → Convert to float
- str() → Convert to string
- bool() → Convert to boolean

Example:

```
a = int("100")      # string -> int
b = int(3.99)       # float -> int
c = float(5)        # int -> float
d = str(123)        # int -> string
e = chr(65)         # int -> char
f = float("12.55")  # string -> float
```

```
print(int(True))    # boolean -> int
```

```
# Unicode conversion
print(ord('a'))     # 97
print(chr(65))      # A
```

```
# octal, hexadecimal, and binary conversion
print(oct(10))      # 0o12
print(bin(10))      # 0b1010
print(hex(10))      # 0xa
```

असतो मा सद्गमय

7. Errors

An error is a mistake in the program that prevents it from executing properly.

Types of Errors:

1. Syntax Error / Compile-Time Error
2. Run-Time Error
3. Logical Error

7.1 Syntax Error / Compile-Time Error:

- A syntax error occurs when you write code that violates Python's grammar rules.
- If Python cannot understand the code, it stops the program before running and shows an error message.
- When we break Python rules such as missing brackets, missing colon, wrong indentation, or wrong keyword spelling, a syntax error occurs.

Example 1: (Missing Colon :)

```
if a > 5
    print("Hello World")
```

Example 2: (Indentation Error)

```
If True:
print("Hi")
```

Example 3: (Unmatched Parentheses)

```
print("Hello World)
```

7.2 Run-Time Errors:

Run-Time Errors are the errors that occur when the program is running.

The syntax of the program is correct, but an unexpected situation happens during execution.

Common Run-Time Errors:

1. ZeroDivisionError
2. IndexError
3. KeyError

4. TypeError
5. ValueError
6. NameError
7. FileNotFoundError
8. AttributeError

1) ZeroDivisionError:

Occurs when you try to divide a number by zero.

Example:

```
a = 10  
b = 0  
print(a / b)
```

2) IndexError:

Occurs when you try to access an index that doesn't exist in a list.

Example:

```
list1 = [1, 2, 3]  
print(list1[5])
```

3) KeyError:

Occurs when you try to access a key that is not present in the dictionary.

Example:

```
my_dict = {"name": "Raj"}  
print(my_dict["age"])
```

4) TypeError:

Occurs when you perform an invalid operation between different data types.

Example:

```
a = "10"  
b = 5  
print(a + b)
```

5) ValueError:

Occurs when the data type is correct but the value is not appropriate.

Example:

```
num = int("abc")
```

6) NameError:

Occurs when you use a variable that is not defined.

Example:

```
print(name)
```

7) FileNotFoundError:

Occurs when a file you are trying to open does not exist.

Example:

```
f = open("data.txt", "r")
```

8) AttributeError:

Occurs when you try to use an attribute or method that does not exist for that object.

Example:

```
x = 10  
x.append(5)
```

7.3 Logical Error:

- A logical error occurs when the program runs successfully (without any syntax or run-time error) but produces the wrong output because the logic used in the code is incorrect.
- Python does not show any error message for logical errors.
- The program runs normally, but the output is not what we expected.
- These errors occur due to wrong formula, wrong condition, or incorrect logic.

Example:

```
a = 10  
b = 5  
print(f'Addition: {a-b}')
```



8. Control Statements

Conditional statements are used to make decisions in a program. They allow the program to execute certain pieces of code only when a specific condition is True.

Why Do We Use Conditional Statements?

In real life, we take decisions based on conditions (for example, if OTP is correct → allow login). Programs also need to behave differently in different situations.

How Do Conditional Statements Work?

Python first evaluates the condition given after if.

The condition gives a Boolean result — either True or False.

- If the condition is True, the indented block of code runs.
- If the condition is False, that block is skipped.

After that, Python executes the remaining program in the normal flow.

Example:

We want to check whether the user has entered the correct password or not:

- If the password is correct → allow login
- Otherwise → reject login

Program:

```
password = input("Enter your password: ")
if password == "12345":
    print("Login Successful!")
```

Types of Conditional Statements:

1. if statement
2. if-else statement
3. if-elif-else statement
4. Nested if statement

8.1 if Statement

- The if statement is used when we want a program to run a specific set of instructions only when a particular condition is True.

- Python first evaluates the expression after the if, converts it into True or False, and then executes the indented block only if the condition is True.
- If the condition is False, Python ignores that block and continues executing the rest of the program.

Syntax of if Statement:

if condition:

 # run if condition is true

Program:

```
age = 20
if age >= 18:
    print("You are Eligible to vote")
print("This statement is independent from if block")
```

Explanation:

Because age >= 18 evaluates to True, Python runs print("You are Eligible to vote") and then continues to execute print("This statement is independent from if block").

8.2 if-else Statement

- In an if-else conditional structure, Python first checks the condition written after the if.
- If this condition evaluates to True, then the code block under the if statement is executed and the else block is skipped.
- If the condition is False, then the code under the else block runs and the if block is ignored.
- Both if and else blocks never run together. Only one of them runs based on the result of the condition.
- The else block does not require a separate condition and runs only when the if condition fails.

Syntax of if-else Statement:

if condition:

 # code runs when condition is True

else:

 # code runs when condition is False

Program:

```
password = "abc123"
if password == "admin123":
    print("Login Successful")
else:
    print("Incorrect Password")
```

Explanation:

Since `password == "admin123"` evaluates to `False`, Python skips the `if` block and executes the `else` block, printing "Incorrect Password".

8.3 if-elif-else Statement

- When multiple conditions are checked in an if-elif-else chain, Python evaluates each condition in order.
- As soon as it finds the first condition that evaluates to `True`, it executes the corresponding block and skips all remaining `elif` and `else` blocks.
- This structure is useful when we want to perform different actions based on more than two possible conditions.
- Only the first matching condition is executed. Even if multiple conditions are `True`, only the first one runs. If none of the conditions are `True`, the `else` block (if present) runs.

Syntax of if-elif-else:

```
if condition1:
    # Code
elif condition2:
    # Code
elif condition3:
    # Code
else:
    # Code (optional)
```

Program:

```
marks = 75
```

```
if marks >= 90:
    print("Excellent")
elif marks >= 75:
    print("Very Good")
elif marks >= 60:
    print("Good")
```

else:

```
print("Needs Improvement")
```

Explanation:

Since marks = 75, the first condition marks >= 90 is False. Python then checks marks >= 75, which is True. Therefore, it executes print("Very Good") and skips all remaining elif and else blocks.

8.4 Nested if Statement

- When one if statement appears inside the indented block of another if statement, the inner if is evaluated only if the outer if condition is True.
- If the outer condition is False, Python does not check the inner if at all.
- This structure is useful when one decision depends on another decision, such as checking login success first and then checking the user role only if login is successful.

Syntax of Nested if:

```
if condition1:  
    # outer if block  
    if condition2:  
        # inner if block  
    else:  
        # inner else block  
else:  
    # outer else block
```

Program:

```
username = "admin"  
password = "1234"  
  
if username == "admin":  
    if password == "1234":  
        print("Login successful")  
    else:  
        print("Incorrect password")  
else:  
    print("Invalid username")
```

Explanation:

Since `username == "admin"` is True, Python enters the outer if block and then checks `password == "1234"`, which is also True. Therefore, it prints "Login successful". If the username had been incorrect, the password check would not have been executed.



9. Iterative Flow (Loops)

What Are Loops?

- A loop is a programming structure that repeats a set of instructions multiple times until a certain condition is met.
- Loops help us avoid writing the same code again and again. For example, if we want to print "Happy Coding" 100 times, instead of writing 100 print statements, we can just use a loop.

Why Do We Need Loops?

- We need loops to automate repetitive tasks — tasks that repeat with slight variation, like printing numbers from 1 to 100, processing items in a list, checking login attempts, etc.
- Without loops, we would write the same lines of code again and again, which is inefficient, error-prone, and hard to manage.

Example:

```
# without loop
print("Ram")
print("Ram")
print("Ram")
```

```
# with loop
for i in range(3):
    print("Ram")
```

How Loops Work?

- Python evaluates a condition (or uses a sequence), and as long as that condition is True, the loop keeps running — repeating the block of code inside it.

- Once the condition becomes False (or all items are iterated), the loop stops automatically and control moves to the next line after the loop.

Types of Loops:

1. for loop
2. while loop
3. Nested loop

9.1 for Loop

- A for loop is used to iterate (repeat) over a sequence like a list, string, tuple, or a range of numbers.
- It helps run a block of code multiple times, once for each item in the sequence.

Syntax of for loop:

for variable in iterable:

code

Example 1:

```
for ch in "Kitti":  
    print(ch)
```

Explanation:

A string is a sequence of characters. So the loop runs once for each character in "Kitti" and prints them one by one.

Example 2:

```
fruits = ["apple", "banana", "mango"]  
for fruit in fruits:  
    print("I like", fruit)
```

Output:

I like apple
I like banana
I like mango

Explanation:

Here, fruits is a list containing 3 items. The loop runs 3 times — each time picking one fruit and printing it with the message.

range() Function

The range() function is used to generate a sequence of numbers.

Syntax:

range(start, stop, step)

- start: optional, default is 0
- stop: required, loop runs till (stop - 1)
- step: optional, default is 1

Example 1:

```
for i in range(2, 11, 2):  
    print(i)
```

Output:

2
4
6
8
10

असतो मा सद्गमय

Explanation:

Start from 2, go till 10 (stop - 1), with step size 2.

Example 2:

```
for i in range(1, 6):  
    print("Number:", i)
```

Output:

```
Number: 1  
Number: 2  
Number: 3  
Number: 4  
Number: 5
```

Explanation:

Here, range(1,6) generates numbers from 1 to 5 (last value is exclusive). Each number is stored in i, and print() runs 5 times.

else with for loop

In Python, else can be used with for loop. The else block executes only when the loop completes naturally, not when it is broken using break.

Example:

```
for i in range(3):  
    print(i)  
else:  
    print("Loop completed!")
```

Output:

```
0
1
2
Loop completed!
```

break Keyword with for Loop

The break statement immediately exits the loop when a specific condition is met.

Example:

```
for i in range(1, 10):
    if i == 5:
        break
    print(i)
```

Output:

```
1
2
3
4
```

Explanation:

As soon as `i == 5`, break stops the loop.

continue Keyword with for Loop

The continue statement skips the current iteration and goes to the next loop cycle.

Example:

```
for i in range(1, 6):  
    if i == 3:  
        continue  
    print(i)
```

Output:

1
2
4
5

Explanation:

When `i == 3`, `print()` is skipped.

pass Statement

The `pass` statement does nothing. It is a placeholder when code is syntactically required but you don't want to write anything yet.

Example:

```
for i in range(5):  
    pass # logic will be added later
```

9.2 While Loop

- A while loop repeatedly executes a block of code as long as the given condition remains True.
- The loop checks the condition before every iteration. If the condition is True, the loop body executes; if it becomes False, the loop immediately stops and control moves to the next line after the loop.
- A while loop usually works with counters and conditions, not necessarily iterables — but it can also be used with lists or strings using indexing.

Syntax of while loop:

while condition:

 # code

Example 1:

```
i = 1
while i <= 5:
    print("Count:", i)
    i += 1
```

Explanation:

Since $i \leq 5$ remains True for values 1 to 5, Python prints each number and increments i until the condition becomes False.

while loop with List (index-based)

Program:

```
fruits = ["apple", "banana", "mango"]
i = 0
while i < len(fruits):
    print(fruits[i])
    i += 1
```

Explanation:

The loop starts from index 0, and because $i < \text{len}(\text{fruits})$ is True for 0, 1, 2, Python prints each fruit using indexing until i becomes 3, where the condition becomes False and the loop stops.

while loop with String

Program:

```
name = "Hari"
i = 0
while i < len(name):
    print(name[i])
    i += 1
```

Explanation:

The loop prints each character of "Hari" one by one because `i < len(name)` is True for indexes 0 to 5.

break with while loop

Program:

```
i = 1
while i <= 10:
    if i == 5:
        break
    print(i)
    i += 1
```

Explanation:

When `i` becomes 5, the `break` statement stops the loop, so only numbers 1 to 4 are printed.

continue with while loop

Program:

```
i = 1
while i <= 5:
    if i == 3:
        i += 1
        continue
```



```
print(i)
i += 1
```

Explanation:

When *i* becomes 3, printing is skipped and the loop moves to the next value, so 3 is not printed.

else with while Loop

- Python allows an else clause to be used with a while loop. This else block runs only when the loop ends normally, i.e., without a break statement.
- If the while loop is exited using a break, then the else block is skipped.

Example:

```
i = 1
while i <= 5:
    if i == 3:
        break
    print("i =", i)
    i += 1
else:
    print("Loop ended normally")
```

Output:

```
i = 1
i = 2
```

Explanation:

As soon as *i* == 3, break was triggered, so the loop ended forcefully and the else block was skipped.

9.3 Nested Loop

- A nested loop means placing one loop inside another loop.
- You can place a for loop inside another for loop, or a while inside a while, or even a for inside a while.
- The inner loop runs completely every time the outer loop runs one time.

Example of Nested for loop

```
for i in range(1, 4):    # Outer loop: 1 to 3
    for j in range(1, 4): # Inner loop: 1 to 3
        print(i, j)
```

Output:

```
1 1
1 2
1 3
2 1
2 2
2 3
3 1
3 2
3 3
```

Explanation:

For every value of i, the inner loop runs completely with values of j. So for i = 1, j goes from 1 to 3, and this repeats for i = 2 and i = 3.

Example of Nested while loop

```
i = 1
while i <= 3:
    j = 1
    while j <= 2:
```

```
print(i, j)
j += 1
i += 1
```

Output:

```
1 1
1 2
2 1
2 2
3 1
3 2
```



10. Strings

- A string in Python is a sequence of characters used to represent textual data like names, messages, sentences, etc.
- It is written inside single ('...'), double ("..."), or triple ("..." or "...") quotes.
- Python considers strings as a data type (class str) and allows various operations on it.

How to Create a String?

You can create strings in Python using different types of quotes.

Example:

```
str1 = 'Hello'
str2 = "Coding"
str3 = """This is
a multi-line
string"""
```

Python automatically understands that text inside quotes is a string. Triple quotes are used for multi-line strings.

String Indexing

Each character in a string has an index number, starting from 0 for the first character. You can access individual characters using square brackets []. Negative indexing allows accessing characters from the end: -1 means last character.

Example:

```
text = "Hello"
print(text[0]) # H
print(text[4]) # o
```

```
print(text[-1]) # o
print(text[-2]) # l
```

String Immutability

In Python, strings are immutable, which means once a string is created, it cannot be changed. If you try to modify a character in a string, Python will show an error. To change it, you have to create a new string.

Example:

```
name = "Ram"
name[0] = 'V' # This will give error
```

String Assignment and Memory Behaviour

Example 1:

```
a = "hello"
b = a
print(id(a), id(b)) # same Addresses (IDs)
```

Python optimizes memory — same strings are stored at one place.

Example 2:

```
a = "hello"
a = "coding"
```

When we assign a new string, Python creates a new object in memory.

String Concatenation

The + operator is used to combine two or more strings.

Example:

```
first = "Hello"
```

```
second = "word"
print(first + " " + second) # Output: Hello word
```

String Repetition

Use * operator to repeat a string multiple times.

Example:

```
print("Hello! " * 3) # Output: Hello! Hello! Hello!
```

Membership Operator with String

You can check if a word/letter exists in a string using in or not in.

Example:

```
msg = "Welcome"
print("come" in msg) # True
print("hello" not in msg) # True
```

String Slicing

You can extract a part of a string using slicing syntax string[start:end]. The end index is not included.

Example:

```
text = "KamalaKhan"
print(text[0:5]) # Kamal
print(text[5:]) # aKhan
print(text[:5]) # Kamal
print(text[-7:-1]) # aKhan
```

Traversing a String

You can go through each character using loops.

Example 1:

```
name = "Ram"
for ch in name:
    print(ch)
```

This loop will print one character per line.

Example 2:

```
name = "Ram"
i = 0
while i < len(name):
    print(name[i])
    i += 1
```

This loop starts with $i = 0$, and continues to run until i is less than the length of the string ($\text{len}(\text{name})$ is 6 in this case).

String Functions

1) len() function

The `len()` function returns the total number of characters in a string, including spaces and special symbols.

```
name = "Qwerty"
print(len(name))    # Output: 6
```

2) capitalize()

Converts only the first character of the string to uppercase and makes all other characters lowercase.

```
text = "hello word"
print(text.capitalize())    # Output: Hello word
```

3) title()

Converts the first letter of every word into uppercase.

```
text = "python with me"  
print(text.title()) # Output: Python With Me
```

4) lower()

The lower() function converts all characters in the string to lowercase.

```
msg = "WELCOME"  
print(msg.lower()) # Output: welcome
```

5) upper()

The upper() function converts all characters to uppercase letters.

```
msg = "welcome"  
print(msg.upper()) # Output: WELCOME
```

6) count()

Counts how many times a specific character or substring appears in the string.

```
text = "Happy"  
print(text.count("p")) # Output: 2
```

7) find()

The find() function returns the index of the first occurrence of a substring. If substring is not found, it returns -1.

```
text = "Vinod Coding"  
print(text.find("Coding")) # Output: 5  
print(text.find("xyz")) # Output: -1
```


8) strip()

Removes spaces from start and end.

```
text = " hello "  
print(text.strip())    # Output: hello
```

9) replace()

Replaces old substring with new substring.

```
msg = "I love Java"  
print(msg.replace("Java", "Python"))    # Output: I love Python
```

10) index()

Returns the index of the first occurrence of a substring. If substring is not found, it throws an error.

```
text = "Vinod Coding"  
print(text.index("C"))    # Output: 5
```

11) endswith()

Returns True if the string ends with the specified suffix.

```
name = "Shyam"  
print(name.endswith("ya"))    # Output: True
```

12) startswith()

Returns True if the string starts with the specified prefix.

```
text = "happy holi"  
print(text.startswith("happy"))    # Output: True
```

13) isalnum()

Returns True if all characters in the string are either letters or digits (no spaces or symbols).

```
text = "Shyamu123"  
print(text.isalnum()) # Output: True
```

14) isalpha()

Returns True if all characters are only letters (no digits, no space).

```
name = "hello hi"  
print(name.isalpha()) # Output: True
```

15) isdigit()

Returns True if the string contains only numbers.

```
num = "2234"  
print(num.isdigit()) # Output: True
```

16) islower()

Returns True if all alphabet characters in the string are lowercase.

```
txt = "happy"  
print(txt.islower()) # Output: True
```

17) isupper()

Returns True if all alphabet characters are uppercase.

```
txt = "HELLO"  
print(txt.isupper()) # Output: True
```

18) lstrip()

Removes white space or specified characters from the beginning (left) of the string.

```
txt = "  hello"  
print(txt.lstrip())    # Output: hello
```

19) rstrip()

Removes white space or specific characters from the end (right) of the string.

```
txt = "hello  "  
print(txt.rstrip())    # Output: hello
```



11. List

- A list is a collection of multiple items stored in a single variable. It is used when we want to group related data together, such as numbers, names, marks, prices, or mixed types. Lists allow us to store large sets of data in an ordered structure where every item has its own position.
- Lists help us manage and process collections of data easily. Instead of creating many variables for multiple values, we store everything in one list. Lists also allow modification, searching, sorting, adding, and removing items—making them extremely flexible.

How to Create a List?

A list is created using square brackets [], and items are separated using commas. A list may contain integers, strings, floats, or even other lists.

Example:

```
Li = [10, "Prishu", True, [1,2], 3.98, {8,7}]
```

Indexing in Lists

- Lists store items in an ordered sequence where each item has an index. Indexing starts from 0.
- Indexing allows us to access items based on their position. For example, `mylist[0]` gives the first item.

Example:

```
fruits = ["apple", "banana", "mango"]  
print(fruits[1]) # banana
```

Negative Indexing

Negative indexing allows accessing items from the end of the list.

`list[-1]` gives the last item.

Mutability of Lists

Lists are mutable, meaning we can change, update, delete, or insert items after the list is created.

Example:

```
nums = [1, 2, 3]
nums[1] = 20
print(nums) # Output: [1, 20, 3]
```

This shows lists are not fixed; they can be modified anytime.

List Slicing

Slicing allows extracting a part (sublist) of the list.

Syntax:

```
list[start : end : step]
```

Slicing helps in copying a portion of the list without changing the original.

Example:

```
nums = [10, 20, 30, 40, 50]
print(nums[1:4]) # [20, 30, 40]
print(nums[:3]) # [10, 20, 30]
print(nums[::2]) # [10, 30, 50]
```

List Operations

1) Concatenation (+)

Combines lists.

```
a = [1, 2]
b = [3, 4]
```

```
c = a + b # [1, 2, 3, 4]
```

2) Repetition (*)

Creates repeated pattern.

```
x = [5] * 3 # [5, 5, 5]
```

3) Membership (in / not in)

Used for searching.

```
fruits = ["apple", "banana"]  
"apple" in fruits    # True  
"mango" not in fruits # True
```

List Methods

1. append()

The append() method adds a single element to the end of the list. It modifies the original list.

```
li = [1, 2, 3, 4, 5]  
li.append(6)  
print(li) # [1, 2, 3, 4, 5, 6]
```

2. insert()

The insert() method adds an element at a specified index. The syntax is insert(index, value), where index is the position where the value will be added.

```
li = [1, 2, 3, 4, 5]  
li.insert(2, 10) # Insert 10 at index 2  
print(li) # [1, 2, 10, 3, 4, 5]
```

3. extend()

The `extend()` method adds all elements of another list (or any iterable) to the end of the current list.

```
li = [1, 2, 3, 4, 5]
li.extend([6, 7, 8])
print(li) # [1, 2, 3, 4, 5, 6, 7, 8]
```

4. remove()

The `remove()` method removes the first occurrence of a specified element from the list. If the element is not found, it raises a `ValueError`.

```
li = [1, 2, 3, 4, 5]
li.remove(3) # Removes the element 3
print(li) # [1, 2, 4, 5]
```

5. pop()

The `pop()` method removes and returns the element at the specified index. If no index is provided, it removes and returns the last item.

```
li = [1, 2, 3, 4, 5]
popped_item = li.pop(2) # Removes and returns the element at index 2
print(popped_item) # 3
print(li) # [1, 2, 4, 5]
```

6. sort()

The `sort()` method sorts the list in ascending order by default. It can also take a `reverse=True` argument to sort in descending order.

```
li = [5, 3, 4, 1, 2]
li.sort()
print(li) # [1, 2, 3, 4, 5]
```

7. reverse()

The `reverse()` method reverses the order of elements in the list.

```
li = [1, 2, 3, 4, 5]
li.reverse()
print(li) # [5, 4, 3, 2, 1]
```

8. count()

The `count()` method counts the number of times a specific element appears in the list.

```
li = [1, 2, 2, 3, 4, 2]
a = li.count(2)
print(a) # 3
```

9. index()

The `index()` method returns the index of the first occurrence of the specified element. If the element is not found, it raises a `ValueError`.

```
li = [1, 2, 3, 4, 5]
i = li.index(3)
print(i) # 2
```

10. clear()

The `clear()` method removes all elements from the list, leaving it empty.

```
li = [1, 2, 3, 4, 5]
li.clear()
print(li) # []
```

List Traversing using a for loop

- A for loop is commonly used to traverse through each element of a list. It iterates through the list one element at a time and allows us to access or modify each element.


```
li = [10, 20, 30]
for num in li:
    print(num)
```

Output:

```
10
20
30
```

List Traversing using a while loop

A while loop can also be used for traversing a list. In this case, the loop continues to run until a certain condition is met, typically the length of the list, and manually keeps track of the index.

```
li = [10, 20, 30]
i = 0
while i < len(li):
    print(li[i])
    i += 1
```

Output:

```
10
20
30
```

असतो मा सद्गमय

12. Tuples

- A tuple is an immutable collection of items. It can hold multiple items in a single variable, similar to a list. However, unlike lists, tuples cannot be changed after they are created. Tuples can store multiple types of data like integers, strings, floats, etc.
- Tuple is immutable – once created, you cannot modify, add, or remove elements.
- Tuple is created using parentheses ().
- We use tuples when we need a collection of items that should not be modified during the program. Since tuples are immutable, they provide safety against accidental changes to data.
- Tuples can store any type of data, including integers, strings, floats, lists, other tuples, and booleans.

Example:

```
Tup = (1, "apple", 3.14, [10, 20], True)
```

Indexing in Tuples

Tuples, like lists, are ordered collections, meaning each element has a position or index. The index starts at 0.

Example:

```
my_tuple = (1, 2, 3, 4, 5)
print(my_tuple[0]) # 1
print(my_tuple[-1]) # 5 (last element)
```

Immutability of Tuples

Tuples are immutable, which means that once you create a tuple, you cannot modify its elements, add new elements, or remove elements. This makes them more efficient for storing data that should remain constant.

Example:

```
my_tuple = (1, 2, 3)
my_tuple[1] = 4 # This will raise a TypeError
```

Tuple Operations

1. Concatenation (+)

You can concatenate two tuples using the + operator.

```
tuple1 = (1, 2)
tuple2 = (3, 4)
result = tuple1 + tuple2
print(result) # (1, 2, 3, 4)
```

2. Repetition (*)

You can repeat the elements of a tuple using the * operator.

```
tuple1 = (1, 2)
result = tuple1 * 3
print(result) # (1, 2, 1, 2, 1, 2)
```

3. Membership (in / not in)

Check if an element exists in the tuple.

```
tuple1 = (1, 2, 3)
print(2 in tuple1) # True
print(4 in tuple1) # False
```

4. Tuple Slicing

Tuples support slicing, just like lists (tuple[start:end:step]).

```
tuple1 = (1, 2, 3, 4, 5)
print(tuple1[1:4]) # (2, 3, 4)
```

Traversing a Tuple using for loop

```
tuple1 = (1, 2, 3, 4, 5)
for item in tuple1:
    print(item)
```

Traversing a Tuple using while loop

```
tuple1 = (1, 2, 3, 4, 5)
i = 0
while i < len(tuple1):
    print(tuple1[i])
    i += 1
```

Tuple Functions

1. len()

The len() function returns the number of elements present in the tuple.

```
tup = (1, 2, 3, 4, 5)
print(len(tup)) # Output: 5
```

2. tuple()

The tuple() function is used to convert an iterable (such as a list, string, or set) into a tuple.

```
li = [1, 2, 3, 4]
tup = tuple(li)
print(tup) # Output: (1, 2, 3, 4)
```

3. count()

The count() function counts how many times a specific element appears in the tuple.

```
tup = (1, 2, 2, 3, 4, 2, 5)
print(tup.count(2)) # Output: 3
```

4. index()

The index() function returns the index of the first occurrence of a specified element. If the element is not found, it raises a ValueError.

```
tup = (10, 20, 30, 40, 50)
print(tup.index(30)) # Output: 2
```

5. sorted()

The `sorted()` function returns a new list that contains the elements of the tuple sorted in ascending order. It does not modify the original tuple.

```
tup = (4, 3, 1, 5, 2)
sorted_tuple = sorted(tup)
print(sorted_tuple) # Output: [1, 2, 3, 4, 5]
```

6. `min()`

The `min()` function returns the smallest element from the tuple.

```
tup = (10, 20, 5, 15)
print(min(tup)) # Output: 5
```

7. `max()`

The `max()` function returns the largest element from the tuple.

```
tup = (10, 20, 5, 15)
print(max(tup)) # Output: 20
```

8. `sum()`

The `sum()` function returns the sum of all numeric elements in the tuple.

```
tup = (1, 2, 3, 4, 5)
print(sum(tup)) # Output: 15
```

असतो मा सद्गमय

13. Sets

- A set is an unordered collection of unique elements. It does not store duplicate values and does not maintain any order of elements. Sets are commonly used when you need to store multiple items and check for the existence of an item efficiently. They are similar to lists and tuples, but they are distinct because they do not allow duplicate values.
- Sets are unordered, meaning elements are not stored in any specific sequence. The order of elements may change.
- Sets are mutable, meaning elements can be added or removed after creation.

How to Create a Set?

A set is created by placing elements inside curly braces {}, with elements separated by commas. Alternatively, you can use the set() constructor.

Example:

```
my_set = {1, 2, 3}
```

```
# Or using the set() constructor
```

```
my_set = set([1, 2, 3])
```

Operations on Sets

1. Adding Elements – add()

The add() function is used to add a single element to a set.

```
my_set = {1, 2, 3}
```

```
my_set.add(4)
```

```
print(my_set) # Output: {1, 2, 3, 4}
```

2. Removing Elements – remove() and discard()

The `remove()` method removes a specific element from the set. If the element does not exist, it raises a `KeyError`.

The `discard()` method also removes a specific element, but if the element does not exist, it does not raise any error.

```
my_set = {1, 2, 3, 4}
my_set.remove(3)
print(my_set) # Output: {1, 2, 4}
```

```
my_set.discard(5) # No error even though 5 is not in the set
```

3. Union – union()

The `union()` method combines two sets and returns a new set with all unique elements from both sets.

```
set1 = {1, 2, 3}
set2 = {3, 4, 5}
set3 = set1.union(set2)
print(set3) # Output: {1, 2, 3, 4, 5}
```

4. Intersection – intersection()

The `intersection()` method returns a new set with the common elements from both sets.

```
set1 = {1, 2, 3}
set2 = {3, 4, 5}
set3 = set1.intersection(set2)
print(set3) # Output: {3}
```

5. Difference – difference()

The `difference()` method returns a new set with elements that are present in the first set but not in the second.

```
set1 = {1, 2, 3}
set2 = {3, 4, 5}
set3 = set1.difference(set2)
print(set3) # Output: {1, 2}
```

6. Symmetric Difference – symmetric_difference()

The `symmetric_difference()` method returns a new set with elements that are in either of the sets, but not in both.

```
set1 = {1, 2, 3}
set2 = {3, 4, 5}
set3 = set1.symmetric_difference(set2)
print(set3) # Output: {1, 2, 4, 5}
```

Traversing a Set using for loop

```
my_set = {10, 20, 30, 40, 50}
for item in my_set:
    print(item)
```

Set Built-in Functions

1. add(x) – Add a Single Element

The `add()` function adds a single element `x` to the set. If the element is already present, it does nothing.

```
my_set = {1, 2, 3}
my_set.add(4)
print(my_set) # Output: {1, 2, 3, 4}
```

2. update(iterable) – Add Multiple Elements

The `update()` function adds multiple elements to the set from an iterable (like a list, tuple, or another set). It can add one or more elements at a time.


```
my_set = {1, 2, 3}
my_set.update([4, 5, 6])
print(my_set) # Output: {1, 2, 3, 4, 5, 6}
```

3. pop() – Removes and Returns an Arbitrary Element

The pop() function removes and returns an arbitrary element from the set. Since sets are unordered, you cannot specify which element is removed. If the set is empty, it raises a KeyError.

```
my_set = {1, 2, 3, 4}
popped_item = my_set.pop()
print(popped_item) # Output: Arbitrary element, e.g., 1
print(my_set)      # Output: Set without the popped element
```

4. clear() – Removes All Elements

The clear() function removes all elements from the set, leaving it empty.

```
my_set = {1, 2, 3, 4}
my_set.clear()
print(my_set) # Output: set()
```

5. copy() – Shallow Copy of the Set

The copy() function returns a shallow copy of the set. This means a new set with the same elements, but modifying the new set will not affect the original set.

```
my_set = {1, 2, 3}
copied_set = my_set.copy()
print(copied_set) # Output: {1, 2, 3}
```

14. Dictionary

- A dictionary is a collection of key-value pairs. Each item in a dictionary has a unique key associated with a corresponding value. Unlike lists or sets, dictionaries are unordered and allow fast lookups by key. Dictionaries are useful for storing data that can be identified by a unique key, such as contact details, student records, etc.

How to Create a Dictionary?

A dictionary is created using curly braces `{}` with key-value pairs, where each key is separated from its value by a colon `:` and the pairs are separated by commas.

Syntax:

```
my_dict = {key1: value1, key2: value2, key3: value3}
```

Example:

```
my_dict = {"name": "Ram", "age": 25, "city": "Jaipur"}  
print(my_dict)
```

How to Access Dictionary Items?

You can access the value of a dictionary by referring to its key inside square brackets `[]` or using the `get()` method.

Example:

```
my_dict = {"name": "Alice", "age": 25, "city": "New York"}
```

```
# Using square brackets
```

```
print(my_dict["name"]) # Output: Alice
```

```
# Using get() method
```

```
print(my_dict.get("age")) # Output: 25
```

Adding or Updating Key-Value Pairs:

```
my_dict = {"name": "Ram", "age": 25}
my_dict["age"] = 26 # Update the value of key "age"
my_dict["city"] = "Jaipur" # Add a new key-value pair
print(my_dict) # Output: {'name': 'Ram', 'age': 26, 'city': 'Jaipur'}
```

Traversing a Dictionary using for loop:

```
my_dict = {"name": "Alice", "age": 25, "city": "New York"}
for key in my_dict:
    print(key)
```

Dictionary Built-in Functions

1. keys() – Returns All Keys

The keys() function returns a view object that displays all the keys in the dictionary.

```
my_dict = {"name": "Ram", "age": 25, "city": "Jaipur"}
keys = my_dict.keys()
print(keys) # Output: dict_keys(['name', 'age', 'city'])
```

2. values() – Returns All Values

The values() function returns a view object that displays all the values in the dictionary.

```
my_dict = {"name": "Ram", "age": 25, "city": "Jaipur"}
values = my_dict.values()
print(values) # Output: dict_values(['Ram', 25, 'Jaipur'])
```

3. items() – Returns Key-Value Pairs

The items() function returns a view object that displays all the key-value pairs in the dictionary.

```
my_dict = {"name": "Ram", "age": 25, "city": "Jaipur"}
items = my_dict.items()
print(items) # Output: dict_items([('name', 'Ram'), ('age', 25), ('city', 'Jaipur')])
```

4. update({...}) – Add/Update Multiple Items

The update() function is used to add multiple items (key-value pairs) to a dictionary. If a key already exists, its value is updated.

```
my_dict = {"name": "Ram", "age": 25}
my_dict.update({"city": "Jaipur", "age": 26})
print(my_dict) # Output: {'name': 'Ram', 'age': 26, 'city': 'Jaipur'}
```

5. pop(key) – Remove by Key

The pop() function removes a specified key and returns its corresponding value. If the key is not found, it raises a KeyError.

```
my_dict = {"name": "Ram", "age": 25, "city": "Jaipur"}
removed_value = my_dict.pop("age")
print(removed_value) # Output: 25
print(my_dict) # Output: {'name': 'Ram', 'city': 'Jaipur'}

my_dict.pop("address") # Raises KeyError
```

6. popitem() – Removes Last Inserted Item

The popitem() function removes and returns the last inserted key-value pair from the dictionary.

```
my_dict = {"name": "Ram", "age": 25, "city": "Jaipur"}
removed_item = my_dict.popitem()
print(removed_item) # Output: ('city', 'Jaipur')
print(my_dict) # Output: {'name': 'Ram', 'age': 25}
```

7. clear() – Empties the Dictionary

The `clear()` function removes all items from the dictionary, leaving it empty.

```
my_dict = {"name": "Ram", "age": 25}
my_dict.clear()
print(my_dict) # Output: {}
```

8. `copy()` – Shallow Copy

The `copy()` function returns a shallow copy of the dictionary. This means a new dictionary is created with the same items, but modifying the new dictionary does not affect the original one.

```
my_dict = {"name": "Ram", "age": 25}
new_dict = my_dict.copy()
print(new_dict) # Output: {'name': 'Ram', 'age': 25}
```

9. `get()` – Retrieve Value for a Key

The `get()` function returns the value for a specific key. If the key is not found, it returns `None` or a default value if provided.

```
my_dict = {"name": "Ram", "age": 25}
print(my_dict.get("name")) # Output: Ram
print(my_dict.get("address", "Not Found")) # Output: Not Found
```

10. `del` – Delete Key-Value Pair

The `del` statement is used to delete a specific key-value pair from the dictionary.

```
my_dict = {"name": "Ram", "age": 25}
del my_dict["age"]
print(my_dict) # Output: {'name': 'Ram'}
```

15. Functions

- A function is a block of reusable code that performs a specific task. It allows us to organize code, avoid repetition, and make programs more modular and maintainable. Functions take input, perform some processing, and may return output. Functions are used to break down a complex problem into smaller, manageable parts.

Why Do We Use Functions?

Code Reusability: Once a function is defined, we can use it multiple times without rewriting the same code.

Modularity: Functions break down a program into smaller, more manageable pieces, making the code easier to understand and maintain.

Avoiding Redundancy: Instead of repeating code, we can call the function wherever necessary.

Improved Readability: Functions improve the readability of the code by organizing it into smaller chunks.

Types of Functions:

1. Built-in Functions
2. User-defined Functions

Built-in Functions: Functions that are already defined in Python, like `print()`, `len()`, `max()`, etc.

User-defined Functions: Functions that are defined by the user to perform specific tasks.

Defining a Function

A function is defined using the `def` keyword, followed by the function name and parentheses `()` containing optional parameters. The function body follows, indented.

Syntax:

```
def function_name(parameters):  
    # Function body  
    return result
```

Example:

```
def greet():  
    print("SKIT")
```

Function Call

To execute a function, you need to call it by its name and pass the required arguments.

Example:

```
def greet(name):  
    print(f'Hello, {name}!')  
  
# Calling the function  
greet("Ram") # Output: Hello, Ram!
```

Arguments

Arguments are the values passed to a function when it is called. They are specified inside the parentheses.

Example:

```
def add(a, b):  
    return a + b
```

```
# Passing arguments  
result = add(5, 3)  
print(result) # Output: 8
```

Here, 5 and 3 are arguments passed to the add() function.

Parameters

Parameters are variables that are defined in the function definition. They accept values (arguments) when the function is called.

Example:

```
def multiply(x, y):  
    return x * y  
  
# Passing arguments  
result = multiply(4, 2)  
print(result) # Output: 8
```

Here, x and y are parameters of the multiply() function.

Default Parameters

A parameter can have a default value. If a value is not provided when the function is called, the default value is used.

Example:

```
def greet(name="Guest"):  
    print(f'Hello, {name}!')  
  
greet() # Output: Hello, Guest!  
greet("Alice") # Output: Hello, Alice!
```

Here, if no argument is passed to the name parameter, it defaults to "Guest".

Returning Values

A function can return a value using the return keyword. Once return is executed, the function terminates, and the value is sent back to the caller.

Example:

```
def square(number):  
    return number * number
```

```
result = square(4)  
print(result) # Output: 16
```

Here, the function `square()` returns the square of the number.

Scope of Variables

The scope of a variable refers to where in the code that variable can be accessed. Variables can have local scope (only within the function) or global scope (accessible throughout the program).

- Local Variable
- Global Variable

Example:

```
x = 10 # Global variable  
  
def fun():  
    y = 20 # Local variable  
    print(x, y) # x is global, y is local  
  
fun() # Output: 10 20  
print(x) # Output: 10  
# print(y) # Error: y is not defined
```

Here, `x` is a global variable that can be accessed anywhere, while `y` is a local variable that is only accessible inside `fun()`.

Recursive Function

A recursive function is a function that calls itself to solve a smaller version of the problem.

How Recursion Works?

- **Base Case:** A condition that stops the recursion. Without a base case, the function would call itself indefinitely, leading to a stack overflow.
- **Recursive Case:** The function calls itself to solve a smaller problem.

Structure of a Recursive Function

```
def fun(parameters):  
    # Base case  
    if condition:  
        return value  
    # Recursive case  
    else:  
        return fun(arguments)
```

Example:

```
def fun(n):  
    if n > 5: # Base case: stop recursion when n exceeds 5  
        return  
    print(n) # Print the current number  
    fun(n + 1) # Recursive call with n+1
```

```
# Calling the function  
fun(1)
```

Output:

```
1  
2  
3  
4  
5
```

Explanation:

The function `fun()` prints the current number `n` and then calls itself recursively with `n + 1` until `n` exceeds 5. The base case is when `n > 5`, which stops further recursion.

`fun(1)` prints 1, then calls `fun(2)`, which prints 2, and so on until `fun(5)` prints 5. Once `n > 5`, the recursion stops.

16. Object-Oriented Programming (OOPs)

- Object-Oriented Programming (OOPs) is a programming paradigm that is based on the concept of objects. It is a way of organizing code to resemble real-world entities. In simple terms, OOPs helps in grouping related data (variables) and functions (methods) into units called objects. These objects represent real-world entities like a "Car," "Student," or "Book." By using OOPs, we can model the behavior and properties of real-world things more easily.
- Before OOPs, code was mostly written as one long list of steps, called procedural programming. This method made it difficult to manage big projects. But with OOPs, we can break the code into small, manageable pieces (objects) that we can use again and again, which makes our code simpler and more organized.
- The main idea of OOPs is to group related data and functions together. For example, in the Car object, we can keep the car's color, model, and year as data (called attributes) and actions like start() and stop() as functions (called methods).

6 Main Pillars of OOPs

- Class
- Object
- Encapsulation
- Inheritance
- Polymorphism
- Abstraction

16.1 Class and Object

Class

A class is like a blueprint or plan for creating objects. It defines the properties and behaviors that objects created from it will have. Just like a house blueprint

shows the structure but is not the real house, a class defines structure but is not an actual object.

Object

An object is an instance of a class. It is the real entity created from the blueprint (class). If Car is a class, then Tesla is an object of that class.

Example of Class and Object

```
# Creating Class
class Student:
    def __init__(self, name, course):
        self.name = name
        self.course = course

    def show(self):
        print(f'Welcome to SKIT, {self.name}!')
        print("Course:", self.course)

# Creating objects
s1 = Student("Aman", "Python")
s2 = Student("Riya", "C++")

s1.show()
s2.show()
```

16.2 Constructor

A constructor is a special method used to initialize the attributes of an object when it is created. It is automatically called when an object is instantiated from a class.

init() Method

The constructor is defined using the `__init__()` method.

Syntax

```
class ClassName:  
    def __init__(self, parameters):  
        self.attribute1 = value1  
        self.attribute2 = value2
```

self Keyword

- self refers to the current object.
- It is automatically passed to every instance method.
- It helps access object variables and methods.

Example

```
class Student:  
    def __init__(self, name, course):  
        self.name = name  
        self.course = course  
  
    def show(self):  
        print(f"Student Name: {self.name}")  
        print(f"Enrolled Course: {self.course}")
```

```
s1 = Student("Aman", "Python")  
s2 = Student("Riya", "C++")
```

```
s1.show()  
s2.show()
```

16.3 Types of Variables

1) Instance Variables

- Bound to a specific object.
- Created when object is instantiated.

- Accessed using self.

2) Class Variables (Static Variables)

- Shared among all objects.
- Defined inside class but outside methods.
- Accessed using class name or object.

Example

```
class Student:
    institute = "SKIT" # Class variable

    def __init__(self, name):
        self.name = name # Instance variable
```

16.4 Types of Methods

1) Instance Method

Works with object data. Uses self.

```
class Student:
    def show(self):
        print("SKIT")
```

2) Class Method

Works with class variables. Uses @classmethod and cls.

```
class Student:
    Institute = "SKIT"
```

```
@classmethod
def info(cls):
    print(cls.Institute)
```

3) Static Method

Does not depend on class or object. Uses @staticmethod.

```
class Student:
    @staticmethod
    def info():
        print("SKIT")
```

16.5 Inheritance

Inheritance allows one class to inherit properties and methods of another class.

- Parent / Super Class
- Child / Sub Class

Single Inheritance

```
class A:
    def show(self):
        print("This is class A")
```

```
class B(A):
    pass
```

```
b1 = B()
b1.show()
```

Multi-level Inheritance

```
class A:
    def Ashow(self):
        print("A class")
```

```
class B(A):
    def Bshow(self):
```

```
print("B class")
```

```
class C(B):  
    def Cshow(self):  
        print("C class")
```

```
c = C()  
c.Ashow()  
c.Bshow()  
c.Cshow()
```

Multiple Inheritance

```
class A:  
    def showA(self):  
        print("A class")
```

```
class B:  
    def showB(self):  
        print("B class")
```

```
class C(A, B):  
    pass
```

```
c = C()  
c.showA()  
c.showB()
```

Hierarchical Inheritance

```
class A:  
    def show(self):  
        print("This is class A")
```



```
class B(A):  
    pass
```

```
class C(A):  
    pass
```

super() Function

Used to call parent class constructor or methods.

```
class A:  
    def __init__(self):  
        print("A class constructor")  
  
class B(A):  
    def __init__(self):  
        super().__init__()  
        print("B class constructor")
```

```
b1 = B()
```

MRO (Method Resolution Order)

Determines which parent method to call in multiple inheritance.

```
class A:  
    def __init__(self):  
        print("A constructor")
```

```
class B:  
    def __init__(self):  
        print("B constructor")
```

```
class C(A, B):  
    def __init__(self):
```

```
print("C constructor")
super().__init__()
```

```
obj = C()
```

16.6 Encapsulation

Encapsulation means wrapping data and methods into a single unit (class).

Types of Members

1. Public
2. Protected (`_variable`)
3. Private (`__variable`)

16.7 Polymorphism

Polymorphism means one name, many forms.

Method Overloading (Compile-Time Style in Python)

```
class Math:
    def add(self, a, b=0, c=0):
        return a + b + c
```

```
m = Math()
print(m.add(10, 20))
print(m.add(10, 20, 5))
```

Method Overriding (Run-Time Polymorphism)

```
class Animal:
    def sound(self):
        print("Animal makes sound")
```

```
class Dog(Animal):
```

```
def sound(self):  
    print("Dog barks")
```

```
class Cat(Animal):  
    def sound(self):  
        print("Cat meows")
```

```
a1 = Dog()  
a2 = Cat()
```

```
a1.sound()  
a2.sound()
```

16.8 Abstraction

Abstraction means showing only important features and hiding internal details.

Python uses the abc module for abstraction.

Abstract Method Example

```
from abc import ABC, abstractmethod
```

```
class Vehicle(ABC):
```

```
    @abstractmethod
```

```
    def start(self):
```

```
        pass
```

```
class Car(Vehicle):
```

```
    def start(self):
```

```
        print("Car starts with key")
```

```
class Bike(Vehicle):
```

```
    def start(self):
```

```
print("Bike starts with button")
```

```
c = Car()
```

```
b = Bike()
```

```
c.start()
```

```
b.start()
```

16.9 Exception Handling

Exception handling prevents program crash due to runtime errors.

Basic Example

```
try:
```

```
    a = 10
```

```
    b = 0
```

```
    print(a / b)
```

```
except:
```

```
    print("Error occurred")
```

```
print("Program continues")
```

Full Syntax

```
try:
```

```
    pass
```

```
except ExceptionType:
```

```
    pass
```

```
else:
```

```
    pass
```

```
finally:
```

```
    pass
```

Multiple Except Example

```
try:  
    a = int("abc")  
    print(10 / a)  
  
except ValueError:  
    print("Value error occurred")  
  
except ZeroDivisionError:  
    print("Cannot divide by zero")  
  
except:  
    print("Some other error")
```

